

Diapositiva 1 — Token / JWT

- ¿Qué es?: token firmado (JWT) que prueba identidad y contiene claims (usuario, rol, expiración).
- Uso en la app: `access_token` enviado en header ``Authorization: Bearer <token>`` en todas las peticiones protegidas.
- Dónde revisar (backend): ``app/services/auth_service.py``, ``app/core/security.py``.
- Dónde revisar (frontend): ``src/store/authStore.js``, ``src/services/apiClient.js``.

****Notas para el expositor:**** mostrar la respuesta de ``POST /auth/login`` que incluye ``access_token``; abrir ``auth_service.py`` y ``authStore.js`` al mismo tiempo.

```
# PowerShell: login y mostrar token
$login = Invoke-RestMethod -Method POST -Uri https://green-saver-api.onrender.com/auth/login -Body (@{email='a
$login.access_token
```

Diapositiva 2 — RBAC (Roles y permisos)

- Idea: cada JWT contiene un claim ``role`` (ej. ``user`` o ``admin``). Backend valida rol antes de endpoints admin.
- Implementación: ``app/core/security.py`` proporciona utilidades; ``app/main.py`` aplica middleware que bloquea rutas bajo ``ADMIN_ONLY_PREFIXES``.
- Demo rápido: llamada a endpoint admin sin token → ``401``; con token de usuario → ``403``; con token admin → ``200``.

****Comando demo (curl):****

```
# Intento sin token -> 401
curl -i https://green-saver-api.onrender.com/admin/quotes

# Con token de usuario -> 403
curl -i -H "Authorization: Bearer <USER_TOKEN>" https://green-saver-api.onrender.com/admin/quotes

# Con token admin -> 200
curl -i -H "Authorization: Bearer <ADMIN_TOKEN>" https://green-saver-api.onrender.com/admin/quotes
```

Diapositiva 3 — Conexión a la Base de Datos

- Dónde están las configuraciones: ``app/db/database.py`` (inicialización) y variables de entorno (``DATABASE_URL``).
- Comportamiento: al iniciar ``uvicorn`` el servicio intenta conectar; errores de conexión aparecen en logs (útil para demostrar credenciales/URL).
- Qué enseñar: variable ``DATABASE_URL`` en entorno de Render o ``.env`` local, y la función de inicialización del DB en ``database.py``.

****Comandos útiles:****

```
# Arrancar backend localmente y seguir logs
cd green-saver-backend
.venv\Scripts\Activate.ps1
uvicorn app.main:app --reload
```

O para ver la variable (Windows PowerShell):

```
Write-Host $env:DATABASE_URL
```

Diapositiva 4 — Flujo de demostración y archivos a mostrar

- Paso 1: Abrir `app/services/auth\_service.py` y `app/core/security.py` (emitir/ver token).
- Paso 2: Abrir `app/main.py` (middleware RBAC) y `app/db/database.py` (conexión DB).
- Paso 3: Ejecutar login y luego llamar a un endpoint protegido (por ejemplo `/quotes`).
- Archivos frontend a abrir: `src/store/authStore.js`, `src/services/apiClient.js`, y `app/(auth)/login.jsx` para mostrar interacción UI.

****Comandos resumen (PowerShell):****

```
# 1) Login -> obtener token
```

```
$login = Invoke-RestMethod -Method POST -Uri https://green-saver-api.onrender.com/auth/login -Body (@{email='a';password='a'})
$token = $login.access_token
```

```
# 2) Llamada a endpoint protegido
```

```
Invoke-RestMethod -Method GET -Uri https://green-saver-api.onrender.com/quotes -Headers @{ Authorization = "Bearer $token" }
```

****Consejo del expositor:**** muestra primero el `access\_token` JSON, luego la cabecera `Authorization` en la petición; alterna entre token de usuario y admin para evidenciar RBAC.

<footer>

Archivo con las diapositivas: docs/sustentacion_diapos.md

</footer>